BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name:

ID:

> **Due Date**
>
> Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

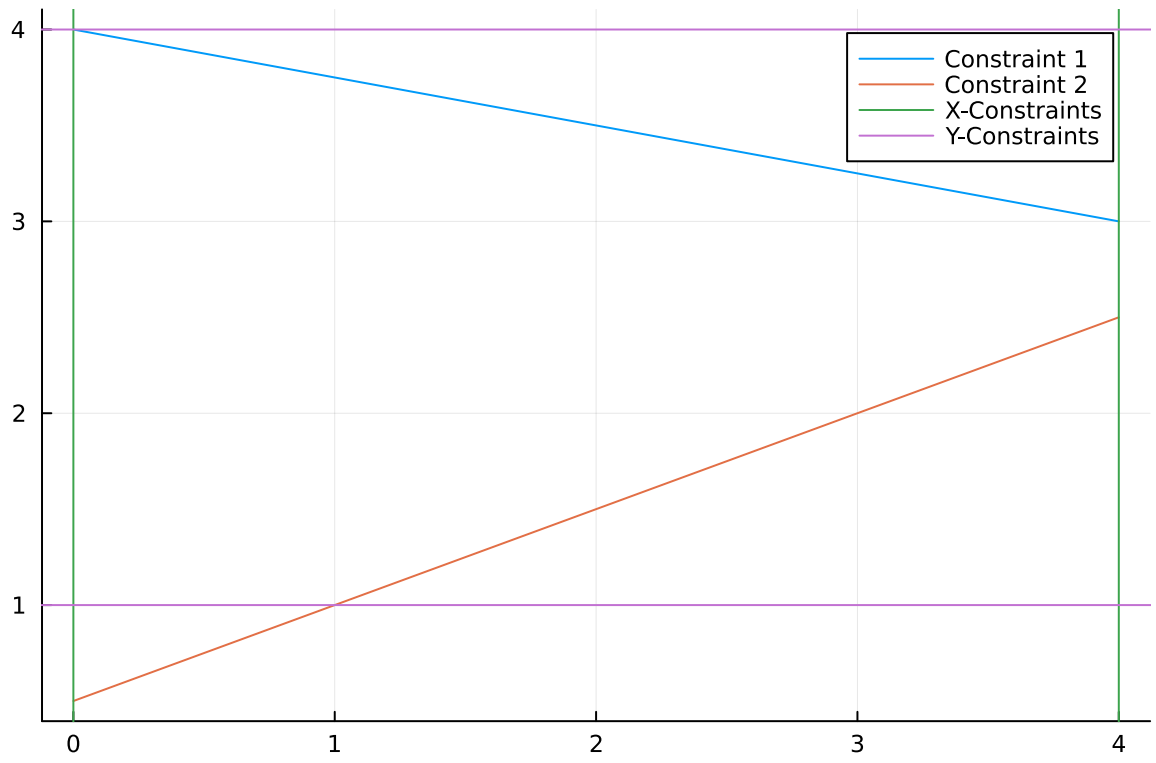
The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
  Activating project at `c:\Users\holly\4750 HW\hw4-ellahollyella`
```

```julia
using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables
```

```julia
# Problem 1
m1 = -1/4
b1 = 4
x1 = 0:0.2:4
y1 = m1 .* x1 .+ b1
m2 = 1/2
b2 = 1/2
x2 = 0:0.2:4
y2 = m2 .* x2 .+ b2
plot(x1, y1, label="Constraint 1")
plot!(x2, y2, label="Constraint 2")
vline!([0, 4], label="X-Constraints")
hline!([1, 4], label="Y-Constraints")
```



Based on these results, there are 4 constraint-intersections that are within the realm of possibilities: the points (0,4), (4,3), (1,1), and (4,2.5). The one that yields the max of 3x+2y is the point (4,3), making that our solution.

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned} \max_{x,y} \quad & 3x + 2y \\ \text{subject to:} \quad & x + 4y \leq 16 \\ & x - 2y \leq -1 \\ & 0 \leq x \leq 4 \\ & 1 \leq y \leq 4. \end{aligned}$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs $70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost ($/ha-yr)	Selling Price ($/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear**.

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

Problem 2 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

Problem 2.1: The decision variables are the pesticide application amount and also the hectacres for each crop. In order to combine these, we have 9 variables, 3 for each crop to cover the 3 different application rates: s0 for the ha of soybeans getting 0 application, s1 for the ha getting 1 kg/ha, and s2 for those getting 2 kg/ha. The other 2 crops follow suit, resulting in 9 decision variables defined in a 1x9 matrix.

Constraints: We are constrained by total ha (130 max), the regulatory limits for pesticide application for each crop (0.8,0.7,and 0.6) and the demand for crops (between 0 and 250,000 kg each). If have variables, s0, s1, and s2, and similarly for w and c, where s(0-2) is total soybean ha (same for other crops), the equations would look as follows:

s0 + s1 + s2 + w0 + w1 + w2 + c0 + c1 + c2 <= 130;

[(s0 * 0) + (s1 * 1) + (s2 * 2)]/s(0-2) <= 0.6,

[(w0 * 0) + (w1 * 1) + (w2 * 2)]/w(0-2) <= 0.7,

[(c0 * 0) + (c1 * 1) + (c2 * 2)]/c(0-2) <= 0.8;

However, these are nonlinear. To make them linear, we move the denominator to the other side, resulting in:

[(s0 * 0) + (s1 * 1) + (s2 * 2)] <= 0.6*s(0-2),

[(w0 * 0) + (w1 * 1) + (w2 * 2)] <= 0.7*w(0-2),

[(c0 * 0) + (c1 * 1) + (c2 * 2)] <= 0.8*c(0-2);

As for production:

0 <= (s0 * 2900)+(s1 * 3800)+(s2 * 4400) <= 250,000, the middle expression will be given the variable name (Ps) for future reference.

0 <= (w0 * 3500)+(w1 * 4100)+(w2 * 4200) <= 250,000, (Pw);

0 <= (c0 * 5900)+(c1 * 6700)+(c2 * 7900) <= 250,000, (Pc);

We also know that all of the decision variables must be non-negative.

The goal is to maximize profits, so we need to use the cost of production, cost of pesticide, and the selling point for each crop to develop our objective function:

(s0 + s1 + s2)*350 + (w0 + w1 + w2)*280 + (c0 + c1 + c2)*390 = cost of prod

70*(1*(s1+w1+c1)+2*(s2+w2+c2)) = cost of pesticide

Ps * 0.36 + Pw * 0.27 + Pc * 0.22 = revenue

profit = revenue - cost of prod - cost of pesticide, and we are maximizing this

```
# 2.2: opt. code
farm_model = Model(HiGHS.Optimizer) # initialize model object
@variable(farm_model, h[1:9] >= 0) # non-negativity constraints
# total prod for each crop
Ps = (2900*h[1]+3800*h[2]+4400*h[3])
Pw = (3500*h[4]+4100*h[5]+4200*h[6])
Pc = (5900*h[7]+6700*h[8]+7900*h[9])
# total area for each crop
hs = sum(h[1:3])
hw = sum(h[4:6])
hc = sum(h[7:9])

@objective(farm_model, Max, ((Ps*0.36+Pw*0.27+Pc*0.22)-(hs*350+hw*280+hc*390)-(70*(1*(h[2]+h
@constraint(farm_model, total, (sum(h)<=130))
# pesticide amount regulatory constraint
@constraint(farm_model, soybean_app, (0*h[1]+1*h[2]+2*h[3])<=0.8*(hs))
@constraint(farm_model, wheat_app, (0*h[4]+1*h[5]+2*h[6])<=0.7*(hw))
@constraint(farm_model, corn_app, (0*h[7]+1*h[8]+2*h[9])<=0.6*(hc))
# Production max constraint
@constraint(farm_model, soybean_prod, (2900*h[1]+3800*h[2]+4400*h[3])<=250000)
@constraint(farm_model, wheat_prod, (3500*h[4]+4100*h[5]+4200*h[6])<=250000)
@constraint(farm_model, corn_prod, (5900*h[7]+6700*h[8]+7900*h[9])<=250000)
# Production nonnegative constraint
@constraint(farm_model, soybean, (2900*h[1]+3800*h[2]+4400*h[3])>=0)
@constraint(farm_model, wheat, (3500*h[4]+4100*h[5]+4200*h[6])>=0)
@constraint(farm_model, corn, (5900*h[7]+6700*h[8]+7900*h[9])>=0);
```

$$5900h_7 + 6700h_8 + 7900h_9 \geq 0$$

```
optimize!(farm_model)
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 10 rows; 9 cols; 36 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
  Cost   [7e+02, 1e+03]
  Bound  [0e+00, 0e+00]
  RHS    [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
7 rows, 9 cols, 27 nonzeros  0s
Presolve : Reductions: rows 7(-3); columns 9(-0); elements 27(-9)
Solving the presolved LP
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0    -1.8170297740e+02 Ph1: 7(28.3601); Du: 9(181.703) 0s
          7    -1.1674116702e+05 Pr: 0(0) 0s
Solving the original LP from the solution after postsolve
Model status        : Optimal
Simplex   iterations: 7
Objective value     :  1.1674116702e+05
P-D objective error :  6.2325284535e-17
HiGHS run time      :          0.00
```

```julia
# Results for 2.2
@show value.(h)
@show objective_value.(farm_model)
```

```
value.(h) = [13.812154696132593, 55.248618784530386, 0.0, 6.743306417339566, 15.7343816404589
objective_value.(farm_model) = 116741.16702082448
```

```
116741.16702082448
```

2.2 writeup

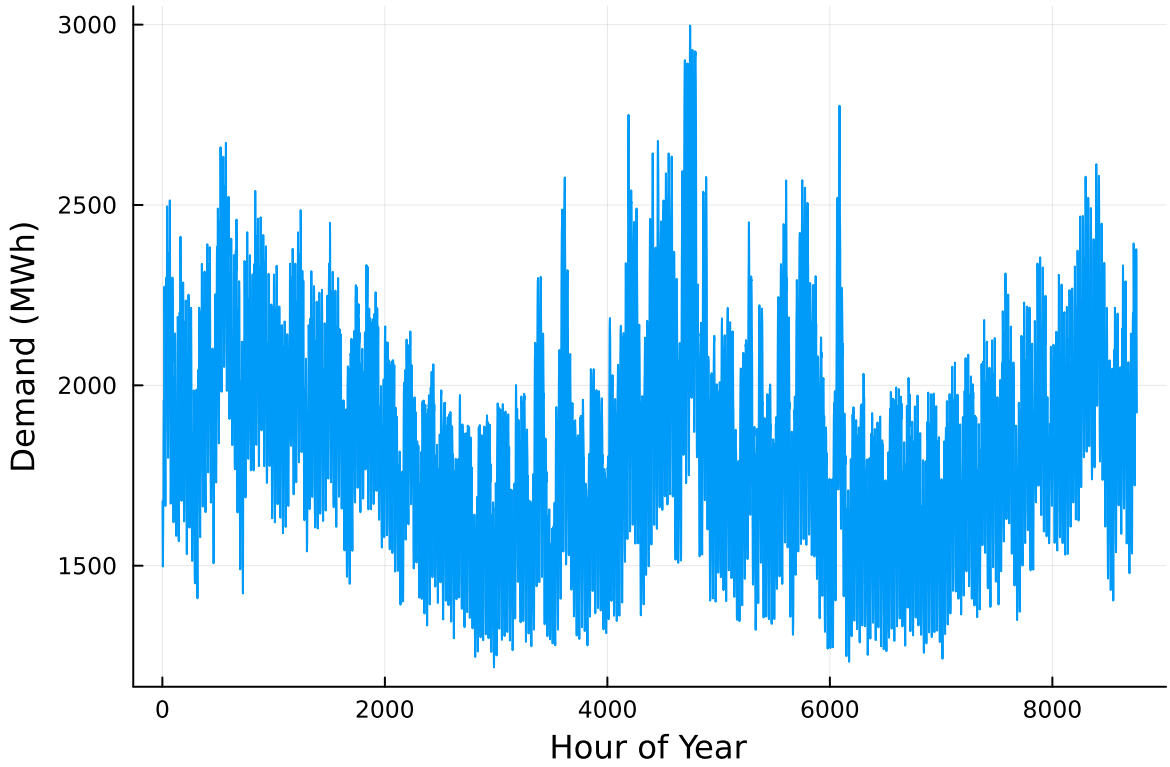
The farmer should plant 69.05 ha of soybean (13.8 no pesticide, 55.25 level 1, 0 level 2), 22.46 ha of wheat (6.73 no pesicide, 15.73 level 1, 0 level 2), and 38.4 ha of corn (26.9 no pesticide, 0 level 1, 11.5 level 2). This will result in a profit of $116,741.17.

```julia
# 2.3
@show shadow_price(total);
```

```
shadow_price(total) = 729.4
```

2.3: Since the shadow price for adding 1ha is 729.4, adding 10ha will result in an increase in profit of $7294.

```julia
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :"Time Stamp" => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs ($/MW installed), variable costs ($/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

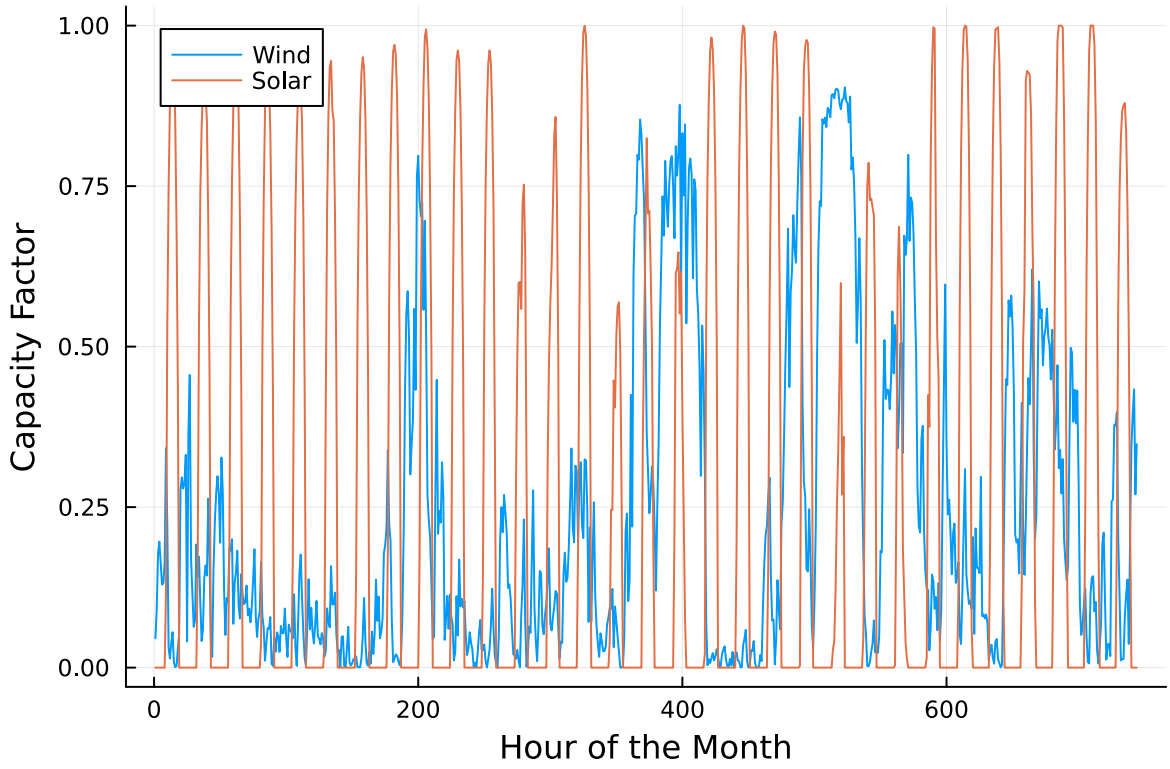
	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

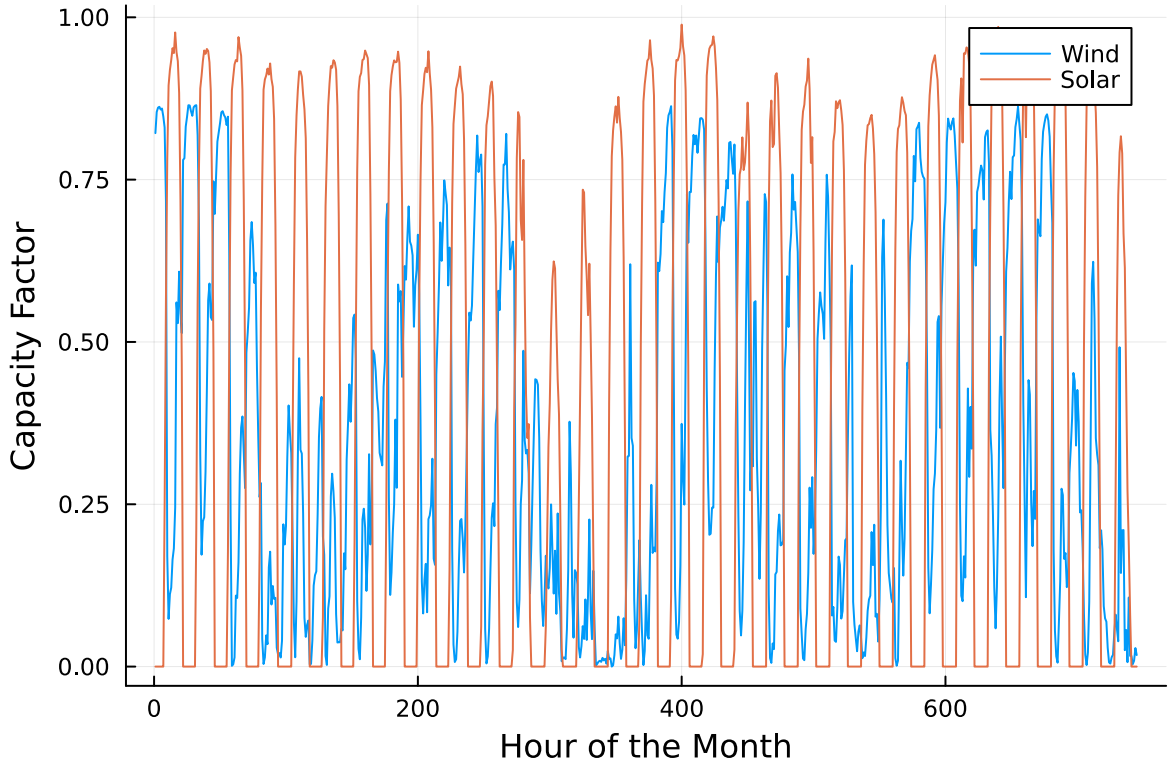
Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```
# load capacify factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

# July Cap factors
p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.

While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month


Wind
Solar
1.00
0.75
0.50
0.25
0.00
Capacity Factor
0
200
400
600
Hour of the Month

the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of $10,000/MWh.

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO2? What is the value to the utility be of allowing it to emit an additional 1000 tCO2/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a `JuMP` variable `x`, but it will return a special container which holds other information about `x` that is useful for `JuMP`. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

References

List any external references consulted, including classmates.

3.1 problem formulation

Our decision variables are the installed capacity of each plant type (MW), (x), the production for each plant (MWh), (y), and the non-served energy (NSE). We want to minimize the total cost, which is the capacity cost based on MW installed, the variable cost (based on production), and the NSE cost, which is $10,000/MWh. Therefore, our objective function will be modeled after lecture to minimize total cost by multiplying capacity for each generator by fixed cost, multiplying generation of each kind by variable cost, and multiplying NSE by $10,000. These 3 numbers will be added together to give us a total to minimize by altering capacity and generation for each plant.

Constraints: We are constrained by meeting demand at every given hour (or facing the consequences of non-served energy), non-negativity, carbon cap, and generation not exceeding capacity for each plant. Now we have to consider a varying capacity factor for solar and wind, and constant capacity factors for the other generators. This will impact our generation/capacity constraint, and we will not be able to exceed the capacity times the Cf. As for carbon, the sum of all carbon emissions from all generation must not exceed 1.5 Mtons, so we will have to take this into consideration as well.

```
#3.1 code for LP
using JuMP, HiGHS
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))
gens = DataFrame(CSV.File("data/generators.csv"))
NSEcost = 10000
G = 1:nrow(gens)
T = 1:nrow(demand);
# cap factor for renewables, need to include nonrenewables too
# all need to be a vector of length 8760
cf_wind = cap_factor.Wind;
```

```
cf_solar = cap_factor.Solar;
cf_geo = fill(0.85, length(cf_wind))
cf_coal = fill(1, length(cf_wind))
cf_ccgt = fill(1, length(cf_wind))
cf_ct = fill(1, length(cf_wind))
# order them how they are presented in a matrix
cf = vcat(
    permutedims(cf_geo),
    permutedims(cf_coal),
    permutedims(cf_ccgt),
    permutedims(cf_ct),
    permutedims(cf_wind),
    permutedims(cf_solar)
)
renewcap = Model(HiGHS.Optimizer)

# decision vars and non-negativity constraints
@variables(renewcap, begin
    x[g in G] >=0
    y[g in G, t in T] >= 0
    NSE[t in T] >= 0
end)
@objective(renewcap, Min, sum(gens[G, :FixedCost] .* x) + sum(gens[G, :VarCost] .* sum(y[:, t
@constraint(renewcap, load[t in T], sum(y[:,t]) + NSE[t] >= demand.Demand[t]);
@constraint(renewcap, availability[g in G, t in T], y[g,t] <= x[g]*cf[g,t]);
@constraint(renewcap, emissions, sum(gens[G, :Emissions] .* sum(y[:,t] for t in T)) <= 150000
```

```
optimize!(renewcap)
```

```
Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 61321 rows; 61326 cols; 188256 nonzeros
Coefficient ranges:
  Matrix [5e-05, 1e+00]
  Cost   [2e+01, 4e+05]
  Bound  [0e+00, 0e+00]
  RHS    [1e+03, 2e+06]
Presolving model
56857 rows, 56862 cols, 179328 nonzeros  0s
56854 rows, 56859 cols, 179322 nonzeros  0s
Presolve : Reductions: rows 56854(-4467); columns 56859(-4467); elements 179322(-8934)
Solving the presolved LP
```

```
Using EKK dual simplex solver - serial
  Iteration        Objective     Infeasibilities num(sum)
          0     0.0000000000e+00 Pr: 8760(4.11193e+06) 0s
      20094     4.3522154332e+08 Pr: 10063(1.15033e+06); Du: 0(8.40301e-06) 5s
      24319     6.6829965944e+08 Pr: 9503(3.52381e+06); Du: 0(5.45033e-06) 11s
      28527     7.5749844164e+08 Pr: 3670(755512); Du: 0(3.55585e-06) 16s
      32688     7.7943426813e+08 Pr: 4477(491368); Du: 0(3.91412e-06) 21s
      36380     7.8535154049e+08 Pr: 60(38.2272); Du: 0(7.56171e-06) 27s
      36414     7.8535295894e+08 Pr: 0(0); Du: 0(1.45306e-12) 27s
Solving the original LP from the solution after postsolve
Model status        : Optimal
Simplex   iterations: 36414
Objective value     :  7.8535295894e+08
P-D objective error :  1.3357583179e-14
HiGHS run time      :         26.95
```

3.2

Our results are as follows:

Installed Capacity (MW):

Geo: 285.6, Coal: 0, CCGT: 1509.2, CT: 703.1, Wind: 2548.9, Solar: 2103.7.

The total cost will be $785 Million, and NSE will be 332.3 MWh.

3.3 (and work for 3.2)

```
cap = value.(x)
gen = value.(y)
nse = sum(value.(NSE))
total_cost = objective_value.(renewcap)

# total cost of installation
TC_separated = value.(x) .* gens[:, :FixedCost]
TC = sum(TC_separated)
display(cap)
display("Total Cost: $total_cost")
display("Total Cost of installation: $TC")
display("Total NSE: $nse")
```

```
# annual generation
total_gen = sum(gen);
geo_gen = sum(gen[1,:]);
coal_gen = sum(gen[2,:]);
ccgt_gen = sum(gen[3,:]);
ct_gen = sum(gen[4,:]);
wind_gen = sum(gen[5,:]);
solar_gen = sum(gen[6,:]);

# % of generation
pct_geo = 100*geo_gen/total_gen
pct_coal = 100*coal_gen/total_gen
pct_ccgt = 100*ccgt_gen/total_gen
pct_ct = 100*ct_gen/total_gen
pct_wind = 100*wind_gen/total_gen
pct_solar = 100*solar_gen/total_gen;
display("% geo gen: $pct_geo")
display("% coal gen: $pct_coal")
display("% ccgt gen: $pct_ccgt")
display("% ct gen: $pct_ct")
display("% wind gen: $pct_wind")
display("% solar gen: $pct_solar")
# % of installed cap
total_cap = sum(cap)
pct_geo_cap = 100*cap[1]/total_cap
pct_coal_cap = 100*cap[2]/total_cap
pct_ccgt_cap = 100*cap[3]/total_cap
pct_ct_cap = 100*cap[4]/total_cap
pct_wind_cap = 100*cap[5]/total_cap
pct_solar_cap = 100*cap[6]/total_cap;

display("% geo cap: $pct_geo_cap")
display("% coal cap: $pct_coal_cap")
display("% ccgt cap: $pct_ccgt_cap")
display("% ct cap: $pct_ct_cap")
display("% wind cap: $pct_wind_cap")
display("% solar cap: $pct_solar_cap")
```

```
1-dimensional DenseAxisArray{Float64,1,...} with index sets:
```

```
    Dimension 1, 1:6
And data, a 6-element Vector{Float64}:
  285.5681352306725
    0.0
 1509.1728698942882
  703.0618976373823
 2548.8879233664907
 2103.7493968268914

"Total Cost: 7.8535295894349e8"

"Total Cost of installation: 6.771681183357971e8"

"Total NSE: 332.2907137925934"

"% geo gen: 6.775952323419224"

"% coal gen: 0.0"

"% ccgt gen: 20.300911049936303"

"% ct gen: 0.791036107394069"

"% wind gen: 40.417867774667364"

"% solar gen: 31.714232744568157"

"% geo cap: 3.9937140417437025"

"% coal cap: 0.0"

"% ccgt cap: 21.106013375921243"

"% ct cap: 9.8324281542873"

"% wind cap: 35.64658739728441"

"% solar cap: 29.421257030763343"
```

write-up for 3.3

The percentages of installed capacity and generation are related and similar. The main differences stem from the CT plant and the geothermal plant, with CT having the greatest difference (0.8% generated vs 10% installed). This may be due to the low installation cost but high variable cost. We need to have a certain amount installed for the periods of extreme demand, especially when wind and solar are not available (night time, etc.). But, we don't want to use this technology unless we must due to the high per MWh cost, resulting in a lower share of generation.

```
# 3.5
@show shadow_price(emissions)
```

```
shadow_price(emissions) = -182.77193609185866
```

```
-182.77193609185866
```

3.5: This is for relaxing the emissions constraint by 1 ton. Therefore, relaxing it by 1000t would result in a reduction of $182,772 in cost.